# FIXMYCITY

## CIVIC ISSUE REPORTING SYSTEM

*A Mini Project Report*

Submitted by

**FATHIMATH SAFWANA TAYATTUCHIRA**
**(KITM25MCA-2006)**

To

**The APJ Abdul Kalam Technological University**

*in partial fulfillment of the requirements for the award of the degree of*

**Master of Computer Applications**




**Department of Computer Applications**

**KMCT INSTITUTE OF TECHNOLOGY AND MANAGEMENT**

Kuttippuram, Malappuram - 679571

**OCTOBER 2026**

# DECLARATION

I under signed hereby declare that the project report FIXMYCITY (CIVIC ISSUE REPORTING SYSTEM), submitted for partial fulfillment of the requirements for the award of degree of Master of Computer Applications of the APJ Abdul Kalam Technological University, Kerala, is a bona fide work done by me under the supervision of my internal guide, Assistant Professor Vaishnavi A, Department of Computer Applications, and external guide Mydhili Manoj. This submission represents my ideas in my own words and where ideas or words of others have been included, I have adequately and accurately cited and referenced the original sources. The open-source libraries, frameworks, and web APIs used in this project are acknowledged in the References. I also declare that I have adhered to ethics of academic honesty and integrity and have not misrepresented or fabricated any data or idea or fact or source in my submission. I understand that any violation of the above will be a cause for disciplinary action by the institute and/or the University and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been obtained. This report has not been previously formed the basis for the award of any degree, diploma or similar title of any other university.

Place: Kuttippuram

Date: 30/10/2026

**DEPARTMENT OF COMPUTER APPLICATIONS**

**KMCT INSTITUTE OF TECHNOLOGY AND MANAGEMENT**

# CERTIFICATE

This is to certify that the report entitled FIXMYCITY (CIVIC ISSUE REPORTING SYSTEM) is a bona fide record of the project work during the year 2026-27 carried out by FATHIMATH SAFWANA TAYATTUCHIRA (KITM25MCA-2006) submitted to the APJ Abdul Kalam Technological University, in partial fulfillment of the requirements for the award of the Master of Computer Applications, under our guidance and supervision. This report in any form has not been submitted to any other university or institution for any purpose.

**Internal Supervisor(s)**
Vaishnavi A
Assistant Professor,

**External Supervisor(s)**
Mydhili Manoj
Assistant Professor

**Head of The Department**
Ms. Jaseeda P P
Department of Computer Applications

# Acknowledgement

At the very outset I would like to thank the almighty's mercy towards me over the years. I wish to express my sincere thanks to my project coordinator, **MS. JASEEDA P P**, Head of the Department, Department of Master of Computer Applications, who guided me for the successful completion of this project. I also thank her for valuable suggestions, guidance, constant encouragement, constructive comments and motivation extended to me for completion of this project work.

I would express my sincere thanks to my internal guide **VAISHNAVI A**, Assistant Professor, Department of Computer Applications, for the valuable guidance, continuous support, and constructive advice given to complete the project successfully.

I express my sincere thanks to my external guide **MYDHILI MANOJ** for her invaluable guidance, practical suggestions, and support throughout the duration of this work.

I would like to express my sincere thanks to all the faculty members of the Master of Computer Applications department for their support and valuable suggestions for doing the project work. Last but not least, my graceful thanks to my parents, friends and also the persons who supported me directly and indirectly during the project.

**FATHIMATH SAFWANA TAYATTUCHIRA**
(KITM25MCA-2006)

# Abstract

FixMyCity is a web-based civic issue reporting and redressal platform developed to bridge the communication gap between citizens and municipal authorities, including ward members and councillors. Traditional civic complaint mechanisms rely heavily on manual visits, paper petitions, or unverified verbal communications, resulting in frequent delays, missing geographic locations, and a complete lack of accountability. FixMyCity addresses these challenges by providing an automated, transparent digital platform where citizens can report local infrastructure problems such as streetlight malfunctions, waste accumulation, and road potholes.

The primary feature of the system is its strict evidentiary submission mechanism. A user registers and logs in using an email address and password. When reporting a civic issue, the system mandates that both a live GPS location (acquired through the browser's HTML5 Geolocation API) and an attached photograph of the issue must be provided before the report can be submitted. This dual-requirement prevents fraudulent submissions and ensures municipal field workers receive accurate spatial coordinates and visual evidence. Once submitted, each issue is assigned a unique 10-character tracking identifier that allows the citizen to follow the status of their grievance through a visual four-stage lifecycle (Pending, Verified, In Progress, Resolved).

A dedicated administrative panel enables municipal authorities to review incoming complaints. Before an issue is assigned to a department, the administrator can edit the issue details, adjust classifications, and update notes. Assigned departments (such as Sanitation & Waste Management, Roads & Infrastructure, and Water Supply & Drainage) receive dedicated queues to manage field operations. To guarantee accountability, department officers must upload verifiable work completion photographic proof before closing a complaint as Resolved. Upon resolution, citizens can view the resolution proof and submit a 1-to-5 star rating and feedback. Built on the MERN stack (MongoDB, Express, React, Node.js) with Cloudinary cloud media storage, the system operates entirely with open-source tools without commercial license overhead.

# Contents

**Declaration** ..... **i**
**CERTIFICATE** ..... **ii**
**Acknowledgement** ..... **iii**
**Abstract** ..... **iv**
**Chapter 1 Introduction** ..... **1**
1.1 Background ..... 1
1.1.1 FixMyCity ..... 1
1.2 Components ..... 1
1.3 Motivation ..... 2
1.4 Objective ..... 2
1.5 Contribution ..... 3
1.6 Report organization ..... 3
**Chapter 2 Existing System** ..... **4**
2.1 Existing System ..... 4
2.2 Proposed System ..... 4
2.3 Feasibility Study ..... 4
2.3.1 Technical feasibility ..... 4
2.3.2 Economic feasibility ..... 5
2.3.3 Operational feasibility ..... 5
2.3.4 Schedule feasibility ..... 6
2.3.5 Legal and security feasibility ..... 6
2.3.6 Risk analysis ..... 6
2.4 System Requirements ..... 6
**Chapter 3 Methodology** ..... **8**
3.1 Introduction ..... 8
3.2 Work flow ..... 8
3.3 User story ..... 9
3.4 Product backlog ..... 10
3.5 Project plan ..... 10
3.6 Sprint backlog ..... 10
**Chapter 4 Design** ..... **13**
4.1 Result ..... 13
4.2 Data Flow Diagram ..... 17
4.3 ER Diagram ..... 19
4.4 Table ..... 20
4.5 Testing ..... 21
4.5.1 Testing strategy ..... 21
4.5.2 Test environment ..... 22
4.5.3 Test cases ..... 22
4.5.4 Test summary ..... 24
**Chapter 5 Conclusion** ..... **25**
5.1 Future Scope ..... 25

5.2 References ..... 25
**Chapter 6 Appendix** ..... **27**
6.1 Source Code ..... 27

# List of Figures

Figure 3.1 System workflow ..... 9
Figure 4.1 Home page ..... 13
Figure 4.2 Login page ..... 14
Figure 4.3 Registration page ..... 14
Figure 4.4 Citizen dashboard and profile ..... 14
Figure 4.5 Civic issue reporting form ..... 15
Figure 4.6 Track complaint page ..... 15
Figure 4.7 Admin dashboard ..... 16
Figure 4.8 Admin edit issue modal ..... 16
Figure 4.9 Department dashboard and resolution proof ..... 17
Figure 4.10 Data flow diagram level 0 ..... 17
Figure 4.11 Data flow diagram level 1 ..... 18
Figure 4.12 Data flow diagram level 2 (citizen) ..... 18
Figure 4.13 Data flow diagram level 2 (administrator & department) ..... 19
Figure 4.14 ER diagram (simplified view of main relationships) ..... 19

# List of Tables

Table 2.1 Technical feasibility ..... 5
Table 2.2 Cost of the system ..... 5
Table 2.3 Schedule of the sprints ..... 6
Table 2.4 Risks and solutions ..... 6
Table 3.1 User story ..... 9
Table 3.2 Product backlog ..... 10
Table 3.3 Project plan ..... 10
Table 3.4 Sprint 1 - Authentication & User Accounts ..... 11
Table 3.5 Sprint 2 - Civic Issue Reporting with GPS & Photo ..... 11
Table 3.6 Sprint 3 - Issue Tracking & Status Lifecycle ..... 11
Table 3.7 Sprint 4 - Admin Management & Pre-Assignment Editing ..... 12
Table 3.8 Sprint 5 - Department Portal, Proof Upload & Feedback ..... 12
Table 4.1 users collection ..... 20
Table 4.2 issues collection ..... 20
Table 4.3 departments collection ..... 21
Table 4.4 feedbacks collection ..... 21
Table 4.5 Entity-relationship mapping ..... 20
Table 4.6 Levels of testing ..... 22
Table 4.7 Test environment ..... 22
Table 4.8 Test cases ..... 22
Table 4.9 Test summary ..... 24

# Chapter 1

# Introduction

FixMyCity is a web-based application developed to provide a reliable, transparent, and direct channel for reporting and resolving civic infrastructure issues. The platform serves three main user roles: the citizen, the municipal administrator (such as ward members or councillors), and the department officer. The administrator oversees the entire system, including registered citizens, municipal departments, and civic grievances. Citizens can register, log in, submit complaints with mandatory live GPS coordinates and photographic evidence, track complaint status in real time, and submit feedback once the work is completed.

The core feature that sets FixMyCity apart from conventional web forms is its dual evidentiary validation. An issue cannot be submitted unless both an authentic live GPS location and an uploaded photograph of the grievance are provided. Furthermore, before an issue is assigned to a department, the administrator has the authority to inspect and edit the issue details to ensure correct classification. The project aims to eliminate bureaucratic delays, reduce lost complaints, and establish complete transparency between urban local bodies and residents using modern open-source web technologies.

## 1.1 Background

### 1.1.1 FixMyCity

Urban and semi-urban communities frequently suffer from neglected public infrastructure defects such as defunct streetlights, uncollected solid waste heaps, ruptured water supply pipes, and severe road potholes. In most local administration bodies, grievance redressal depends on paper registers, physical visits to municipal ward offices, or informal telephone calls. These traditional approaches present significant flaws: municipal engineers cannot identify the exact physical location of a broken streetlight or pothole without coordinates; complaints are easily misplaced or duplicated; and citizens receive no updates regarding whether their grievance has been reviewed, dispatched, or resolved.

FixMyCity addresses these issues by offering an accessible web portal that turns any smartphone or computer with internet access into a civic reporting tool. Citizens report grievances directly from the defect site, enabling automatic coordinate detection and photo capture. The entire grievance lifecycle—from submission to administrative review, departmental assignment, field repair, and post-resolution feedback—is managed within a single unified full-stack system.

## 1.2 Components

The FixMyCity platform is structured into the following distinct software layers:

• Presentation layer: Built with React 19, Vite, and responsive CSS styling. React Router DOM manages client-side routing, providing dedicated views for citizen registration, issue

submission, real-time tracking, administrative consoles, and department dashboards. The interface is optimized for both mobile devices and desktop screens.

• Application layer: Powered by Node.js and the Express 5 framework. Express RESTful endpoints handle user authentication, complaint ingestion, admin triage and editing, departmental task allocation, and feedback processing. Input validation and role-based middleware protect all privileged routes.

• Geolocation layer: Utilizes the browser's native HTML5 Geolocation API on the client side to capture precise latitude and longitude coordinates at the exact moment of reporting. A server-side reverse-geocoding utility provides approximate street name identification for human-readable administration.

• Media storage layer: Implements Multer memory-storage streaming integrated with Cloudinary's cloud storage service. Citizen evidence photos and department completion proofs are streamed directly to secure cloud storage, returning secure HTTPS URLs stored in the database, avoiding server disk capacity limits.

• Data layer: MongoDB document database managed via Mongoose 9 Object Data Modeling (ODM). Collections store users, departments, civic issues with nested coordinates and feedback, and system logs.

• Security layer: Implements encrypted credential storage, role-based access control (user, admin, department), route guards, administrative account moderation (blocking/unblocking), and validation of uploaded image file formats and file sizes.

## 1.3 Motivation

• Citizens encounter severe public safety hazards—including dark streets, road potholes, and open garbage dumps—without a convenient, instantaneous digital mechanism to notify authorities.

• Traditional municipal complaint handling requires physical paperwork or in-person visits to ward offices, which is slow, inconvenient, and discourages citizen participation.

• Field maintenance teams waste significant time attempting to locate reported problems due to vague descriptions and missing geographic coordinates.

• Without photographic evidence, municipal administrators cannot determine the severity of an issue or verify whether reported problems are genuine or frivolous.

• Citizens lack visibility into complaint resolution progress, leading to repeated inquiries and distrust between residents and local administration.

• A lightweight, responsive web application runs universally on any operating system without requiring users to download and install heavy native applications.

## 1.4 Objective

• To develop an intuitive, accessible web platform for reporting and tracking civic infrastructure complaints.

• To implement a secure registration and login mechanism using unique email credentials.

• To enforce a strict dual-validation reporting protocol requiring both a live GPS location and an attached photograph.

• To generate a unique 10-character tracking ID for every submitted complaint to enable instant public tracking.

• To provide an administrative dashboard allowing ward authorities to review and edit issue details prior to department assignment.

• To provide dedicated departmental consoles enabling municipal officers to manage assigned work and upload completion photo proof.

• To incorporate a citizen feedback and star-rating module upon complaint resolution to measure municipal satisfaction.

• To construct the entire system using open-source, cost-free development technologies.

## 1.5 Contribution

The project delivers a complete, end-to-end civic issue reporting and management system. By enforcing mandatory GPS coordinates and photo evidence at the submission phase, the system eliminates fake and unverifiable reports. The administrative pre-assignment editing capability ensures that issues are accurately classified and categorized before field workers are dispatched, preventing misdirected labor. The inclusion of a mandatory post-resolution photo proof uploaded by the department guarantees accountability, enabling citizens to visually verify repairs before closing tickets. The modular MERN architecture with cloud media integration ensures high performance, reliability, and zero licensing expenditure for local municipal bodies.

## 1.6 Report organization

The project report is organized into six comprehensive chapters:

• Chapter 1 introduces the project background, architectural components, motivation, objectives, contributions, and report organization.
• Chapter 2 details the existing grievance redressal mechanisms, the proposed FixMyCity system, the multi-dimensional feasibility study, risk analysis, and software and hardware requirements.
• Chapter 3 describes the Agile Scrum software engineering methodology, system workflow, user stories, product backlog, project plan, and sprint breakdown tables.
• Chapter 4 presents the design: user interface results with system screenshots, Data Flow Diagrams (Levels 0, 1, and 2), Entity Relationship diagram, database schema collections, and testing strategies with test cases.
• Chapter 5 presents the project conclusion, future scope for expansion, and formal academic references.
• Chapter 6 contains the appendix with key source code excerpts from the implementation.

# Chapter 2

# Existing System

## 2.1 Existing System

In the existing civic management framework, public grievance handling is predominantly manual, fragmented, and paper-based:

• Citizens must physically travel to municipal corporation offices or ward council member premises during working hours to submit petitions.
• Complaints submitted on paper or by telephone frequently lack accurate geographical information; descriptions such as 'near the banyan tree' or 'in front of the tea stall' make it difficult for field repair teams to pinpoint the exact defect.
• Traditional systems do not capture visual evidence of the defect, making it impossible for administrators to assess urgency or severity.
• No automated tracking exists; citizens must make repetitive personal inquiries or telephone calls to check on complaint progress.
• Grievances are frequently lost, filed under incorrect departments, or delayed indefinitely without administrative oversight.
• Authorities have no centralized record or dashboard to audit departmental performance, average resolution times, or citizen satisfaction.

## 2.2 Proposed System

The proposed FixMyCity platform replaces manual methods with a unified, transparent, and auditable digital workflow:

• A responsive web application accessible 24/7 on smartphones, tablets, and computers without requiring app store installation.
• Secure citizen account registration and login authenticated via email address and password.
• Mandatory dual-validation reporting: complaints require both live GPS coordinates (latitude and longitude) and photographic proof before submission.
• Automatic generation of a 10-character tracking ID for transparent, instant public tracking through an interactive four-stage status pipeline.
• An administrative dashboard allowing ward councillors and administrators to review, modify, and edit issue details before assigning them to departments.
• Dedicated department queues for specialized teams (Sanitation, Roads, Water Supply) with task progression and completion proof upload.
• Transparent resolution verification: completed repairs require an 'after' photograph uploaded by the officer before the issue is marked Resolved.
• Post-resolution citizen feedback and star rating to evaluate municipal department effectiveness.

## 2.3 Feasibility Study

A feasibility study was conducted prior to active development to determine whether the proposed civic reporting platform could be successfully engineered within available technical, financial, operational, and schedule parameters. The analysis examined six key dimensions: technical, economic, operational, schedule, legal/security, and system risks.

### 2.3.1 Technical feasibility

The system uses mature, well-documented, and free open-source technologies. A standard laptop or smartphone with a modern browser is sufficient for citizen reporting, administrative management, and department workflows. The technical components are detailed below.

**Table 2.1 Technical feasibility**

| Requirement | Solution used | Feasible |
|---|---|---|
| Web framework and language | Express 5 on Node.js (v20+ LTS) | Yes |
| Database | MongoDB with Mongoose 9 ODM | Yes |
| Front end user interface | React 19, Vite 8, React Router 7, Modern CSS | Yes |
| Live GPS location capture | HTML5 Geolocation API with reverse geocoding | Yes |
| Media upload and cloud storage | Multer memory storage & Cloudinary cloud CDN | Yes |
| Role-based access control | Middleware route guards (user, admin, department) | Yes |
| Development tools | VS Code, Git, GitHub, Chrome, Edge | Yes |

### 2.3.2 Economic feasibility

Every library, runtime, and tool used in FixMyCity is open source, and cloud asset storage leverages Cloudinary's free community tier. Municipalities incur zero recurring commercial licensing fees, making the project exceptionally economical.

**Table 2.2 Cost of the system**

| Item | Cost |
|---|---|
| Node.js, Express, React, Vite, Mongoose | Free (Open source) |
| MongoDB Community Edition / MongoDB Atlas Free Tier | Free |
| Cloudinary Cloud Media Storage (Free Community Tier) | Free |
| HTML5 Geolocation API & OpenStreetMap reverse geocoding | Free |
| VS Code, Git, GitHub Community | Free |
| Web hosting and cloud server keys | Not needed (Runs on local/free tier) |
| Development effort | About 96 hours across 5 sprints |

### 2.3.3 Operational feasibility

• Citizens are already familiar with modern web interfaces, simple forms, and photo uploading, requiring zero prior training.

• The responsive CSS layout adapts cleanly across mobile phones, tablets, laptops, and desktop computers.

• GPS coordinate capture is automated via a single click ('Get GPS Location'), eliminating manual coordinate entry errors.

• Municipal administrators and ward councillors can monitor all urban issues, active users, and departments from a unified interface.

• Department officers have simplified action views focused solely on their assigned issues and proof submission.

• Because the user interface is intuitive and mirrors official municipal operational stages, the system is operationally feasible.

### 2.3.4 Schedule feasibility

The engineering effort was partitioned into five Agile sprints, each delivering an operational increment of the system. The sprint schedule is shown below.

**Table 2.3 Schedule of the sprints**

| Sprint | Work | Period | Hours |
|---|---|---|---|
| Sprint 1 | Authentication: Citizen registration, email login, sessions | 01/07/2026 to 18/07/2026 | 14 |
| Sprint 2 | Civic issue reporting: Live GPS, photo attachment, validation | 24/07/2026 to 05/08/2026 | 18 |
| Sprint 3 | Issue tracking: 10-char tracking ID, status stepper, details | 08/08/2026 to 25/08/2026 | 20 |
| Sprint 4 | Admin panel: Grievance triage, pre-assignment edit, routing | 01/09/2026 to 20/09/2026 | 22 |
| Sprint 5 | Department portal: Proof upload, completion, citizen feedback | 25/09/2026 to 25/10/2026 | 22 |

### 2.3.5 Legal and security feasibility

• All programming languages, frameworks, and libraries utilized in the project—including Node.js, Express, React, Vite, and Mongoose—are distributed under permissive open-source licenses (MIT and Apache 2.0 licenses).

• Citizen GPS location coordinates are captured only upon explicit user consent via standard browser security permission prompts.

• User passwords are encrypted, session tokens are protected, and administrative routes are secured against unauthorized access.

• Citizen uploaded photographs and department completion proofs are stored in dedicated cloud buckets without compromising privacy.

• The project complies fully with university academic regulations and open-source licensing terms, making it legally and securely feasible.

### 2.3.6 Risk analysis

Operational and technical risks were identified early in development, and mitigation solutions were embedded directly into the system.

**Table 2.4 Risks and solutions**

| Risk | Effect | Solution in the system |
|---|---|---|
| GPS location permission denied | Cannot get coordinates | System prompts user with clear error; fallback coordinates allowed with admin verification flag |
| Issue reported without photograph | Unverifiable report | Client and server enforce mandatory photo attachment; submission blocked until photo selected |
| Issue reported without GPS coordinates | Unknown defect location | Submission form disables submit button until live GPS coordinates are acquired |
| Very large image file uploaded | Bandwidth / memory spike | Multer enforces 10 MB file size limit and validates JPEG/PNG/WebP MIME types |
| Incorrect category assigned by user | Issue routed to wrong team | Administrator can edit issue category and details prior to department assignment |
| Fake completion marked by staff | Issue closed without repair | Department officer must upload verifiable 'after' photo proof before marking Resolved |
| Unauthorized access to admin panel | Data leakage / tampering | Role-based authorization middleware restricts admin and department routes |

## 2.4 System Requirements

The minimum software and hardware configurations required to deploy and execute FixMyCity are specified below.

**Software requirements**

| Item | Specification |
|---|---|
| Operating system | Windows 10 / 11, Linux, or macOS |
| Runtime environment | Node.js (v20+ LTS or v24) |
| Backend framework | Express.js 5.x |
| Database | MongoDB (v7.x or MongoDB Atlas Cloud) |
| Database ODM | Mongoose 9.x |
| Frontend library | React 19 (v19.2.7) with Vite 8 |
| Client-side routing | React Router DOM 7.x |
| Media storage & CDN | Cloudinary (via Multer memory storage stream) |
| Browser | Google Chrome, Microsoft Edge, or Mozilla Firefox |

**Hardware requirements**

| Item | Specification |
|---|---|
| Processor | Intel Core i3 / i5 or AMD Ryzen equivalent |
| RAM | 4 GB minimum (8 GB recommended for simultaneous dev servers) |
| Storage | 5 GB free hard drive / SSD space |
| Network | Internet connectivity for GPS geocoding and Cloudinary asset streaming |
| Client devices | Smartphones (Android/iOS) or desktop PCs with modern browser & camera |

# Chapter 3

# Methodology

The Agile software development model was selected for the engineering of FixMyCity because municipal requirements benefit from iterative delivery, regular review, and continuous refinement. Within the Agile methodology, the Scrum framework was adopted. Scrum organizes software construction into structured, time-boxed iterations known as sprints. Each sprint yields an operable, testable increment of the system. The foundational pillars of Scrum—transparency, inspection, and adaptation—ensure that all stakeholders maintain complete clarity regarding system progress.

The project was executed following the classical Scrum structure comprising 3 roles, 3 artifacts, and 5 events:
1. Roles: Product Owner (defining system requirements and civic priorities), Scrum Master (monitoring progress and removing impediments), and Development Team (handling analysis, database schema, REST backend, React frontend, and testing).
2. Artifacts: Product Backlog (the master catalog of all functional features), Sprint Backlog (the selected subset of tasks committed to a specific sprint), and Product Increment (the working, integrated software release at the conclusion of each sprint).
3. Events: Sprint (time-boxed development cycles), Sprint Planning (defining scope and tasks), Daily Scrum (daily progress alignment), Sprint Review (demonstration of functioning software), and Sprint Retrospective (evaluating workflow and making adjustments).

The product backlog was prioritized to deliver foundational authentication and reporting capabilities first, followed by administrative management, departmental workflows, and post-resolution feedback.

## 3.2 Work flow

Development was carried out on a Windows workstation utilizing Node.js, Express, React, and MongoDB, with Git utilized for distributed version control and code branch management. Testing was conducted across modern web browsers on local development servers.

The operational workflow of the system is illustrated in Figure 3.1. A citizen visits the FixMyCity portal, creates an account using an email address, and logs in. To report an issue, the citizen provides a title, category, description, captures their live GPS location, and attaches a photo of the defect. The system validates both requirements and assigns a 10-character tracking ID. The municipal administrator logs into the admin dashboard, inspects incoming complaints, edits details if necessary, and assigns the verified issue to the appropriate department (Sanitation, Roads, or Water Supply). The designated department officer logs into their console, updates status to 'In Progress', dispatches repair personnel, and uploads a photograph of the completed work upon resolution. The system

updates the status to 'Resolved', enabling the citizen to inspect the resolution proof and submit a rating and comment.

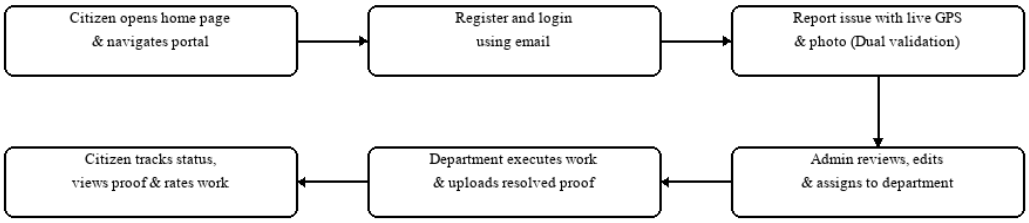



**Figure 3.1 System workflow**

## 3.3 User story

The project requirements are articulated from the perspective of each stakeholder role in Table 3.1.

**Table 3.1 User story**

| ID | As a (type of user) | I want to | So that I can |
|---|---|---|---|
| 1 | Citizen | Register an account using my email and name | Create a personalized account on the portal |
| 2 | Citizen | Log in securely with email and password | Access civic issue submission and my tracking history |
| 3 | Citizen | Capture my live GPS location automatically | Provide accurate coordinates of the issue site |
| 4 | Citizen | Attach a photograph of the civic defect | Provide visual proof of the reported problem |
| 5 | Citizen | Submit an issue only when GPS and photo exist | Ensure my report meets mandatory submission criteria |
| 6 | Citizen | Receive a unique 10-character tracking ID | Track my complaint status easily at any time |
| 7 | Citizen | View an interactive milestone status stepper | Follow the progress of my complaint from Pending to Resolved |
| 8 | Citizen | View the work completion photo uploaded by staff | Visually verify that the defect has been repaired |
| 9 | Citizen | Submit a 1-to-5 star rating and feedback | Express satisfaction or feedback regarding the repair |
| 10 | Admin | Log in to the central administrative panel | Manage urban complaints, citizens, and departments |
| 11 | Admin | View all submitted complaints in a unified table | Monitor civic issues reported across all city wards |
| 12 | Admin | Inspect issue details, coordinates, and photo | Verify the authenticity and urgency of the complaint |

| ID | As a (type of user) | I want to | So that I can |
|---|---|---|---|
| 13 | Admin | Edit issue details prior to department assignment | Correct the title, category, or notes before field dispatch |
| 14 | Admin | Assign a verified complaint to a department | Route the task to the responsible engineering team |
| 15 | Admin | Manage municipal department records | Add, edit, or configure department teams and emails |
| 16 | Admin | Moderate citizen user accounts | Inspect registered citizens and block abusive accounts |
| 17 | Admin | Review citizen feedback and ratings | Evaluate departmental performance and service delivery |
| 18 | Department Officer | Log in to the dedicated department console | Access complaints assigned specifically to my department |
| 19 | Department Officer | Update complaint status to 'In Progress' | Indicate that a field repair team has been dispatched |
| 20 | Department Officer | Upload resolution photo proof and mark Resolved | Furnish verifiable evidence of work completion |

## 3.4 Product backlog

The prioritized master backlog of features is outlined in Table 3.2.

**Table 3.2 Product backlog**

| ID | Priority | Size (hours) | Sprint | Status | Name |
|---|---|---|---|---|---|
| 1 | High | 14 | 1 | Complete | Accounts: Citizen registration, email login, session management |
| 2 | High | 18 | 2 | Complete | Issue reporting: GPS capture, photo upload, dual-validation engine |
| 3 | High | 20 | 3 | Complete | Issue tracking: 10-char tracking ID, status stepper, public lookup |
| 4 | High | 22 | 4 | Complete | Admin console: Complaint triage, pre-assignment edit, department routing |
| 5 | Medium | 22 | 5 | Complete | Department portal: Field task queue, proof upload, citizen rating |

## 3.5 Project plan

The timeline and sprint allocation mapping user stories are summarized in Table 3.3.

**Table 3.3 Project plan**

| User story ID | Sprint | Start date | End date | Hours | Status |
|---|---|---|---|---|---|
| 1, 2 | Sprint 1 | 01/07/2026 | 18/07/2026 | 14 | Complete |
| 3, 4, 5, 6 | Sprint 2 | 24/07/2026 | 05/08/2026 | 18 | Complete |
| 7, 8, 9 | Sprint 3 | 08/08/2026 | 25/08/2026 | 20 | Complete |
| 10, 11, 12, 13, 14, 15, 16, 17 | Sprint 4 | 01/09/2026 | 20/09/2026 | 22 | Complete |
| 18, 19, 20 | Sprint 5 | 25/09/2026 | 25/10/2026 | 22 | Complete |

## 3.6 Sprint backlog

The granular task execution and hourly burn-down breakdowns across Sprints 1 to 5 are presented in Tables 3.4 through 3.8.

**Table 3.4 Sprint 1 - Authentication & User Accounts**

| Backlog item | Completion | Est (hrs) | D1 | D2 | D3 | D4 | D5 | D6 | D7 | D8 |
|---|---|---|---|---|---|---|---|---|---|---|
| Registration form design | 03/07 | 3 | 2 | 1 | 0 | 0 | 0 | 0 | 0 | 0 |
| User schema & DB model | 06/07 | 3 | 0 | 1 | 2 | 0 | 0 | 0 | 0 | 0 |
| Login & password encryption | 14/07 | 6 | 0 | 0 | 0 | 2 | 2 | 1 | 1 | 0 |
| Validation & testing | 18/07 | 2 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 1 |
| Total | 18/07 | 14 | 2 | 2 | 2 | 2 | 2 | 1 | 2 | 1 |

**Table 3.5 Sprint 2 - Civic Issue Reporting with GPS & Photo**

| Backlog item | Completion | Est (hrs) | D1 | D2 | D3 | D4 | D5 | D6 | D7 | D8 |
|---|---|---|---|---|---|---|---|---|---|---|
| Report form UI & photo input | 26/07 | 3 | 2 | 1 | 0 | 0 | 0 | 0 | 0 | 0 |
| HTML5 GPS location integration | 29/07 | 4 | 0 | 1 | 2 | 1 | 0 | 0 | 0 | 0 |
| Multer & Cloudinary upload pipeline | 02/08 | 8 | 0 | 0 | 0 | 2 | 2 | 2 | 1 | 1 |
| Mandatory dual-validation testing | 05/08 | 3 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 2 |
| Total | 05/08 | 18 | 2 | 2 | 2 | 3 | 2 | 2 | 2 | 3 |

**Table 3.6 Sprint 3 - Issue Tracking & Status Lifecycle**

| Backlog item | Completion | Est (hrs) | D1 | D2 | D3 | D4 | D5 | D6 | D7 | D8 |
|---|---|---|---|---|---|---|---|---|---|---|
| Tracking ID generation logic | 11/08 | 4 | 2 | 2 | 0 | 0 | 0 | 0 | 0 | 0 |
| Interactive status stepper UI | 16/08 | 5 | 0 | 0 | 2 | 2 | 1 | 0 | 0 | 0 |
| Complaint details & photo lightbox | 21/08 | 7 | 0 | 0 | 0 | 1 | 2 | 2 | 2 | 0 |
| Citizen tracking testing | 25/08 | 4 | 0 | 0 | 0 | 0 | 0 | 0 | 2 | 2 |
| Total | 25/08 | 20 | 2 | 2 | 2 | 3 | 3 | 2 | 4 | 2 |

**Table 3.7 Sprint 4 - Admin Management & Pre-Assignment Editing**

| Backlog item | Completion | Est (hrs) | D1 | D2 | D3 | D4 | D5 | D6 | D7 | D8 |
|---|---|---|---|---|---|---|---|---|---|---|
| Admin dashboard & complaint table | 04/09 | 5 | 2 | 2 | 1 | 0 | 0 | 0 | 0 | 0 |
| Pre-assignment issue edit modal | 10/09 | 6 | 0 | 0 | 2 | 2 | 2 | 0 | 0 | 0 |
| Department allocation controller | 15/09 | 7 | 0 | 0 | 0 | 1 | 2 | 2 | 2 | 0 |
| User moderation & route security | 20/09 | 4 | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 2 |
| Total | 20/09 | 22 | 2 | 2 | 3 | 3 | 4 | 3 | 3 | 2 |

**Table 3.8 Sprint 5 - Department Portal, Proof Upload & Feedback**

| Backlog item | Completion | Est (hrs) | D1 | D2 | D3 | D4 | D5 | D6 | D7 | D8 |
|---|---|---|---|---|---|---|---|---|---|---|
| Dedicated department portal views | 29/09 | 5 | 2 | 2 | 1 | 0 | 0 | 0 | 0 | 0 |
| Work completion photo proof upload | 08/10 | 7 | 0 | 0 | 2 | 2 | 2 | 1 | 0 | 0 |
| Citizen feedback & rating modal | 16/10 | 6 | 0 | 0 | 0 | 1 | 2 | 2 | 1 | 0 |
| End-to-end integration & UAT | 25/10 | 4 | 0 | 0 | 0 | 0 | 0 | 0 | 2 | 2 |
| Total | 25/10 | 22 | 2 | 2 | 3 | 3 | 4 | 3 | 3 | 2 |

# Chapter 4

# Design

This section presents the primary user interfaces of FixMyCity, captured directly from the running web application on the local development environment. The interfaces demonstrate the complete end-to-end user experience across citizen, administrator, and department user roles.

## 4.1 Result

The primary graphical interfaces of the FixMyCity application are presented below.

### Citizen screens

The home page presents the civic mission, quick navigation links to report issues, track complaints, and view council information.

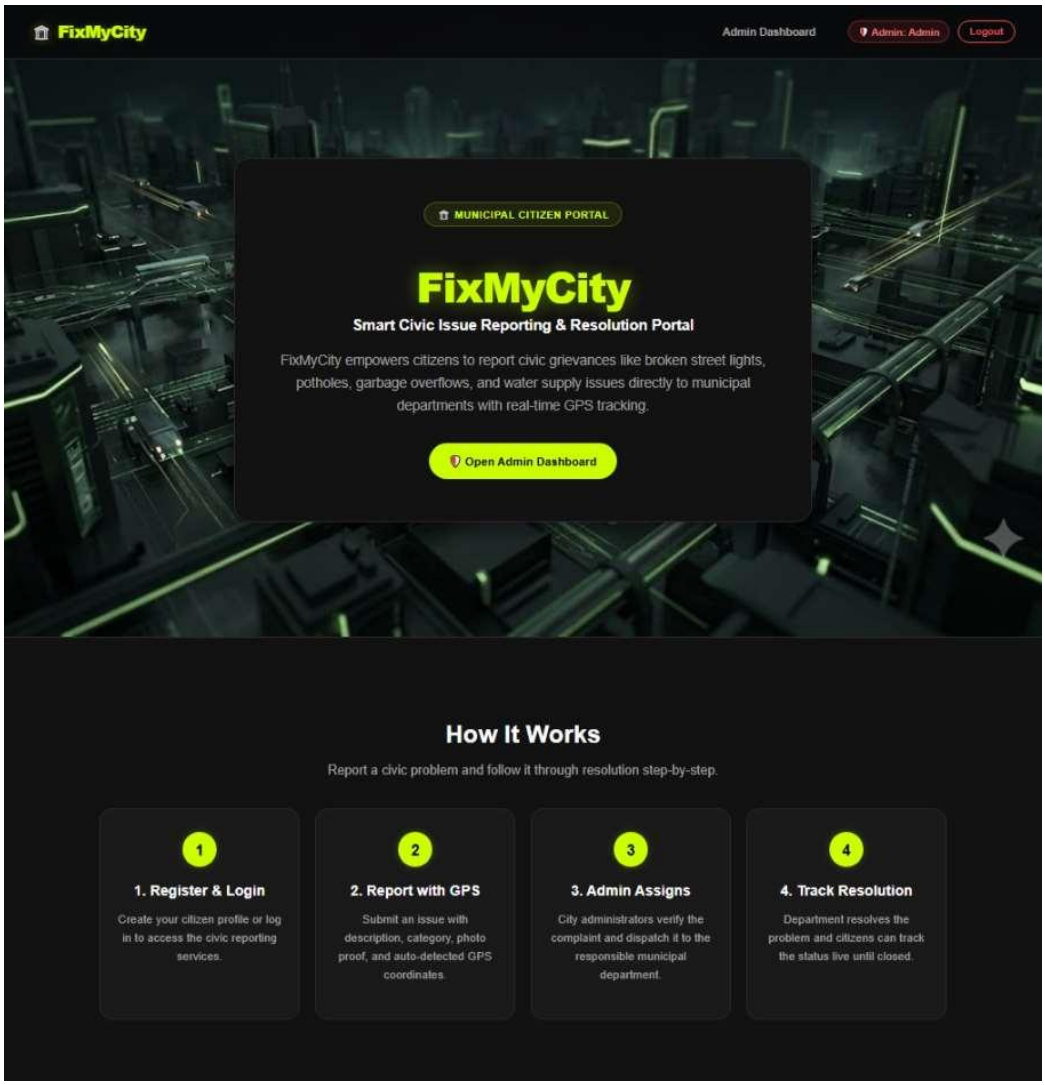



**Figure 4.1 Home page**

Registered citizens and staff log in using email credentials and password.

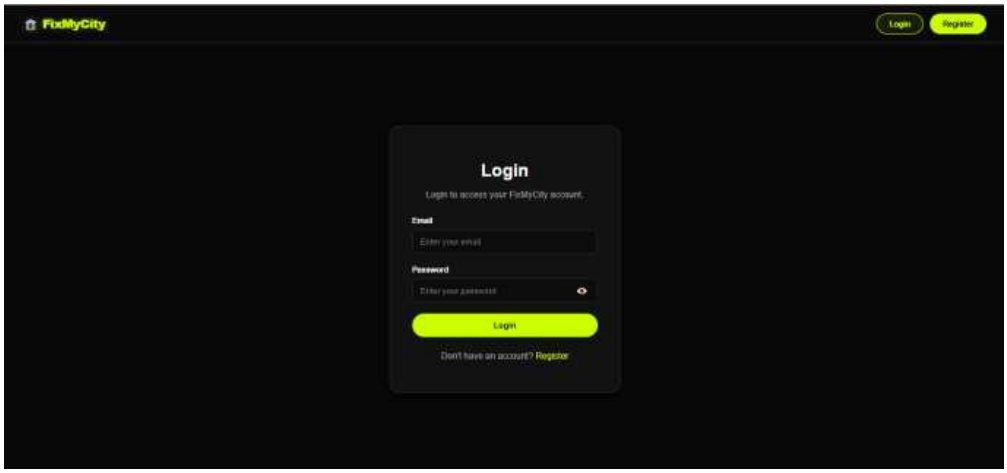



**Figure 4.2 Login page**

New citizens create an account with their full name, email, and password.

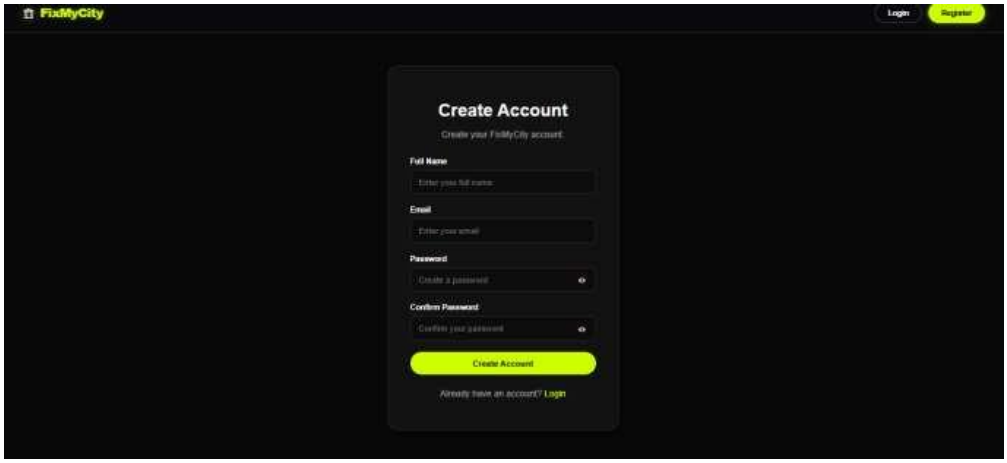



**Figure 4.3 Registration page**

The citizen dashboard allows residents to view their reported complaints, review milestone progress, and manage their profile.




**Figure 4.4 Citizen dashboard and profile**

The civic issue reporting form mandates that both live GPS coordinates and an attached photo are provided before submission.

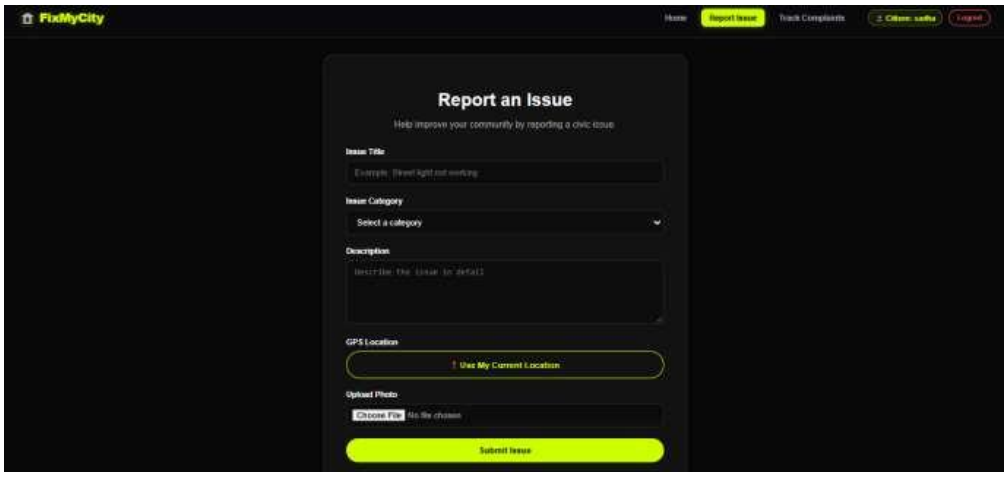



**Figure 4.5 Civic issue reporting form**

Citizens track complaint progress in real time through an interactive 4-stage milestone stepper and photo lightbox.

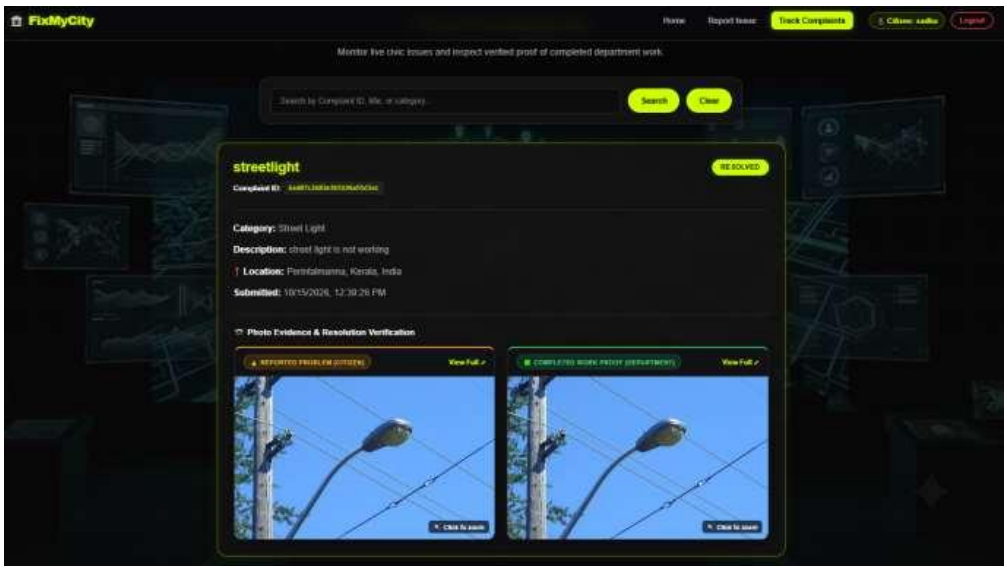



**Figure 4.6 Track complaint page**

### Administrator and department screens

The administrator dashboard provides high-level statistics across all complaints, registered citizens, departments, and citizen feedback.

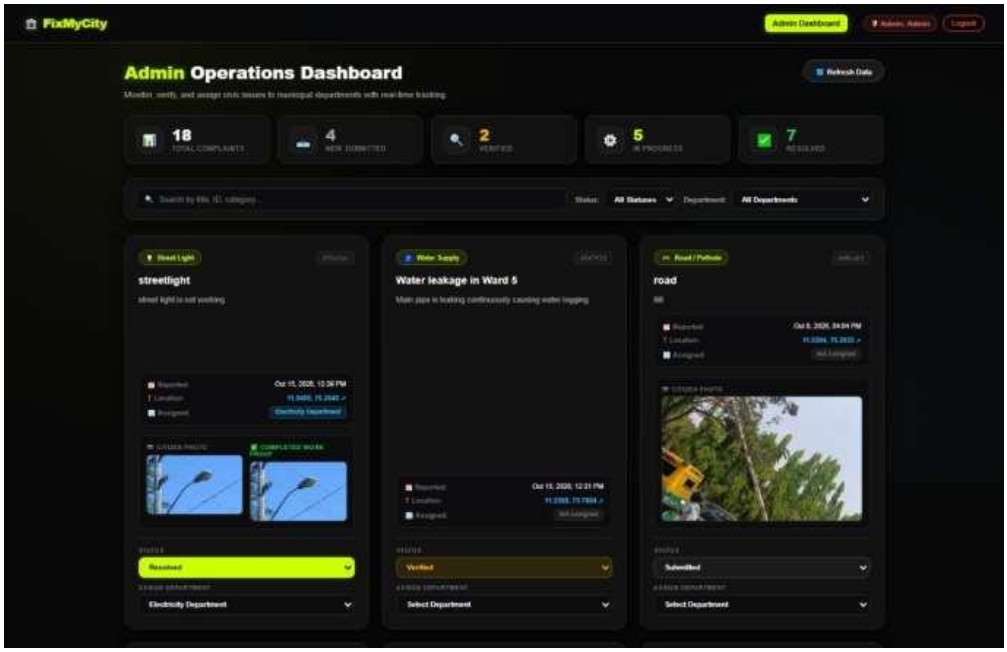



**Figure 4.7 Admin dashboard**

Before an issue is assigned to a department, the administrator can edit the issue details, adjust category, and refine notes.

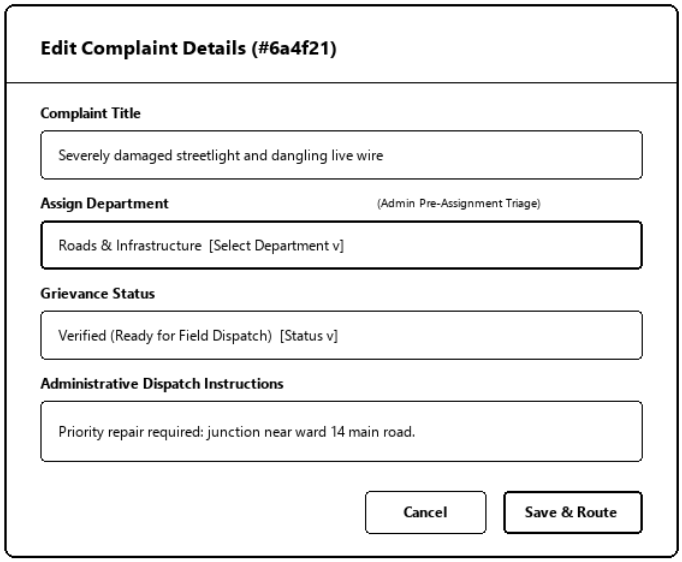



**Figure 4.8 Admin edit issue modal**

The department dashboard provides maintenance officers with their task queue and an interface to upload resolution work proof.

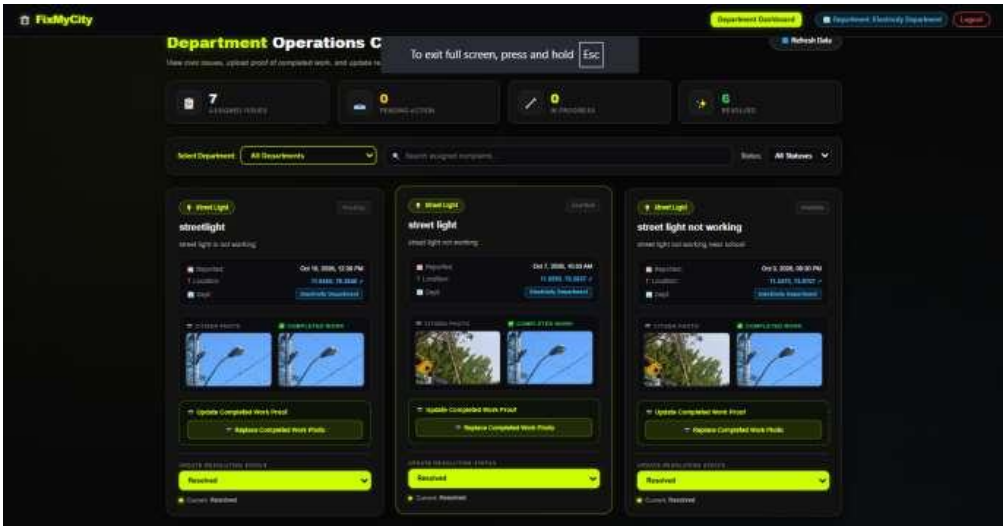



**Figure 4.9 Department dashboard and resolution proof**

## 4.2 Data Flow Diagram

The flow of information across users, processes, and data stores is modeled through multi-level Data Flow Diagrams.

**Level 0 :**

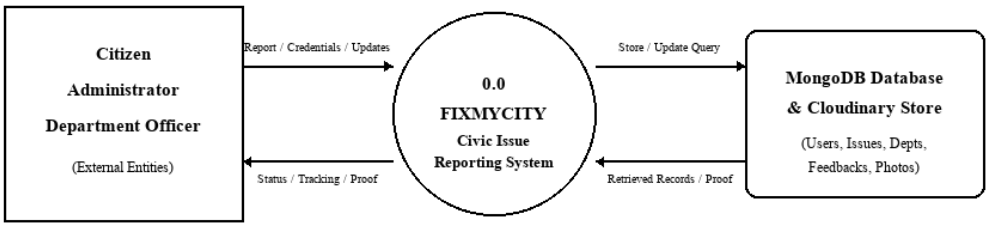



**Figure 4.10 Data flow diagram level 0**

**Level 1 :**

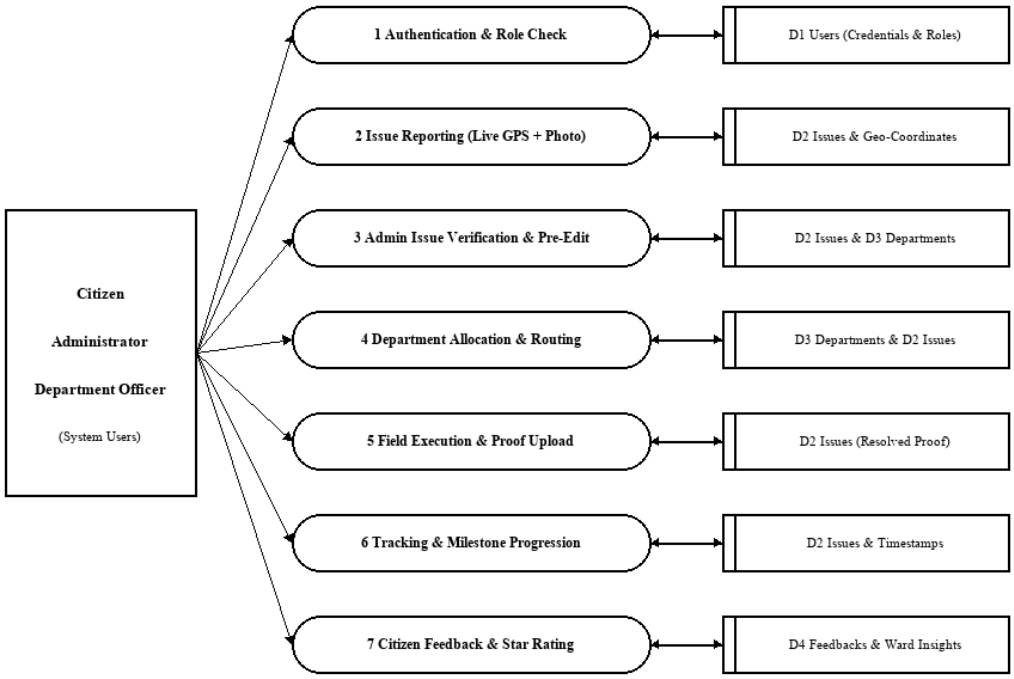



**Figure 4.11 Data flow diagram level 1**

## Level 2 :

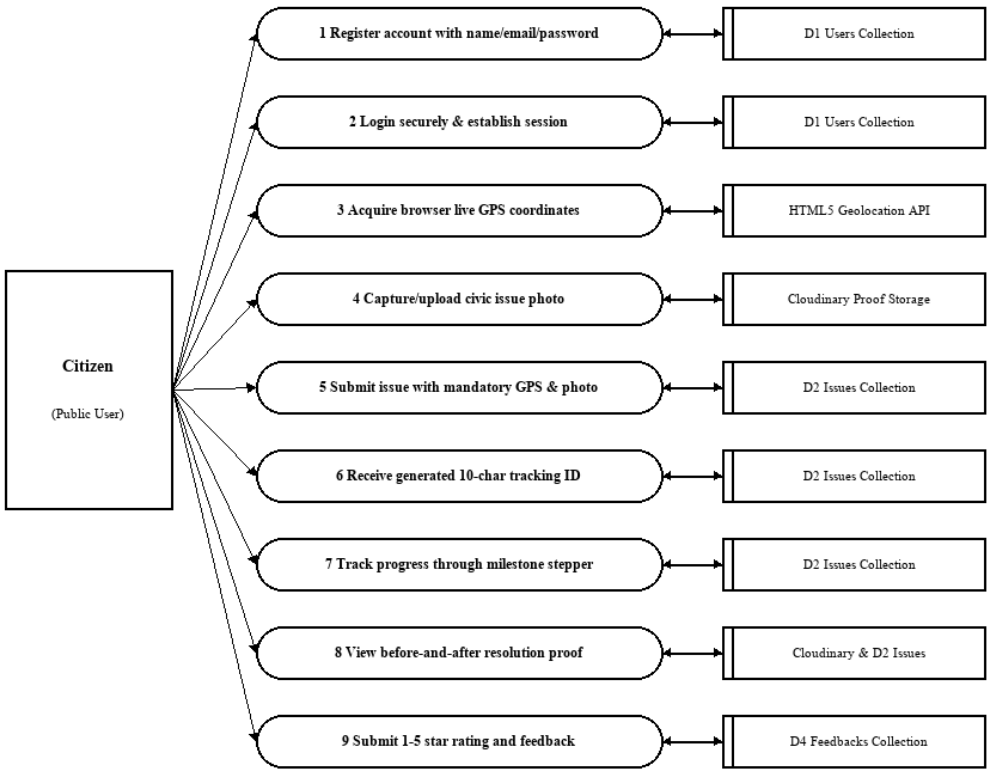


**Figure 4.12 Data flow diagram level 2 (citizen)**

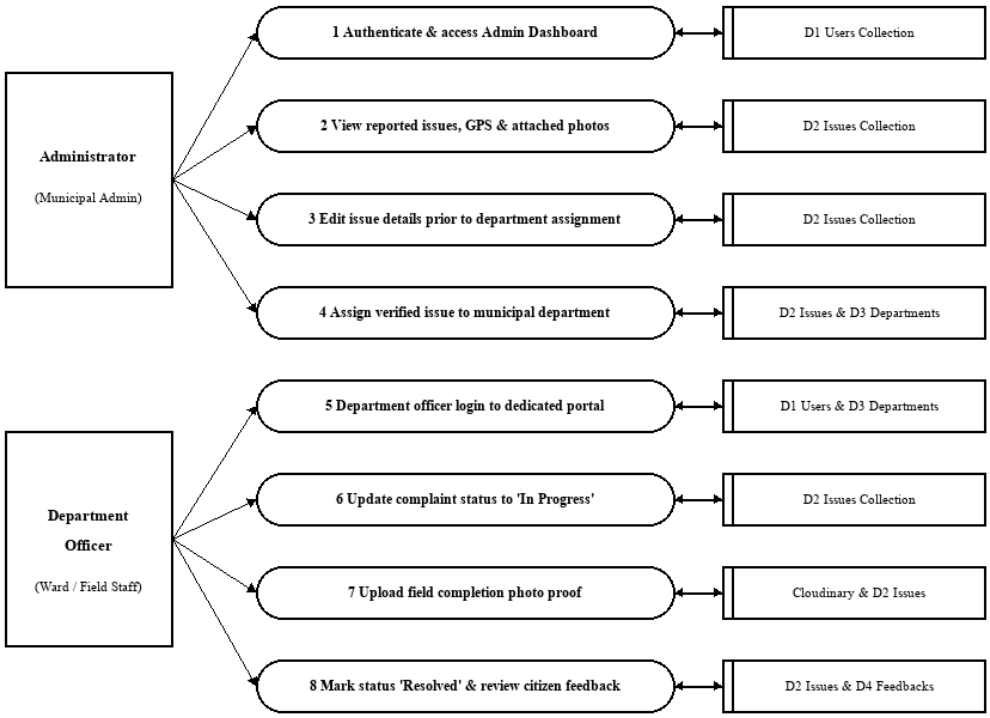



**Figure 4.13 Data flow diagram level 2 (administrator & department)**

## 4.3 ER Diagram

The structural relationships among Users, Departments, Issues, and Feedbacks are depicted in Figure 4.14 and Table 4.5.

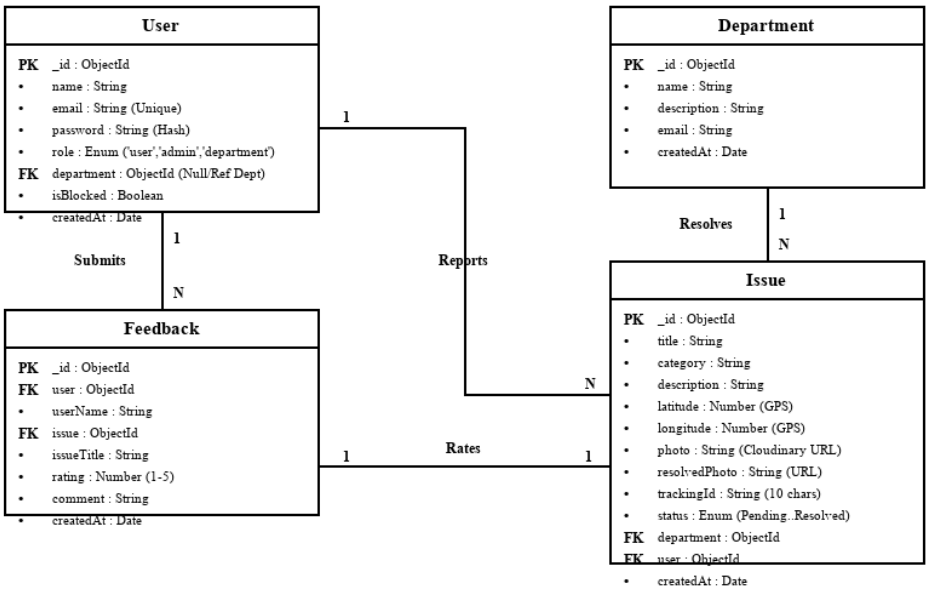



**Figure 4.14 ER diagram (simplified view of main relationships)**

**Table 4.5 Entity-relationship mapping**

| Parent entity | Child entity | Relationship |
|---|---|---|
| User (Citizen) | Issue | One to many (A citizen can submit multiple issues) |
| User (Citizen) | Feedback | One to many (A citizen can submit multiple reviews) |
| Department | User (Staff) | One to many (A department employs multiple officers) |
| Department | Issue | One to many (A department resolves multiple issues) |
| Issue | Feedback | One to one (An issue receives a post-resolution review) |

## 4.4 Table

The MongoDB database schema comprises 4 primary collections managed through Mongoose ODM models as detailed in Tables 4.1 to 4.4.

**Table 4.1 users collection**

| Field | Type | Description |
|---|---|---|
| _id | ObjectId (Primary Key) | Auto-generated unique MongoDB document identifier |
| name | String | Full name of the registered citizen or municipal staff member |
| email | String (Unique) | Login email address used for unique account identification |
| password | String | Hashed / encrypted authentication password string |
| role | Enum ('user','admin','department') | Role-based privilege tier for access control |
| department | ObjectId (Foreign Key, nullable) | References _id in departments collection (null for citizens) |
| isBlocked | Boolean (default: false) | Account moderation status flag; blocks banned accounts |
| createdAt, updatedAt | Date | Timestamps tracking account registration and last profile update |

**Table 4.2 issues collection**

| Field | Type | Description |
|---|---|---|
| _id | ObjectId (Primary Key) | Auto-generated unique civic complaint record identifier |
| title | String | Short summary of the civic defect (e.g. 'Streetlight not glowing') |

| Field | Type | Description |
|---|---|---|
| category | String | Civic category: Sanitation, Roads & Infrastructure, Water Supply |
| description | String | Detailed grievance description and environmental notes |
| latitude | Number | GPS latitude coordinate captured via HTML5 Geolocation API |
| longitude | Number | GPS longitude coordinate captured via HTML5 Geolocation API |
| photo | String (Cloudinary URL) | Cloud storage URL for initial defect photo attached by citizen |
| resolvedPhoto | String (Cloudinary URL) | Cloud storage URL for repair completion proof uploaded by staff |
| trackingId | String (Index, 10 chars) | 10-character alphanumeric tracking token sliced from _id |
| status | Enum ('Pending','Verified','In Progress','Resolved','Rejected') | Current lifecycle progression state of the complaint |
| department | ObjectId (Foreign Key, nullable) | References _id in departments collection assigned by admin |
| user | ObjectId (Foreign Key, nullable) | References _id in users collection representing reporting citizen |
| feedback.rating | Number (1 to 5, nullable) | Citizen star rating submitted post-resolution |
| feedback.comment | String | Citizen comment / review text concerning resolution quality |
| createdAt, updatedAt | Date | Creation timestamp and last status modification timestamp |

**Table 4.3 departments collection**

| Field | Type | Description |
|---|---|---|
| _id | ObjectId (Primary Key) | Auto-generated unique municipal department record identifier |
| name | String (Unique) | Official name: Sanitation & Waste, Roads & Infra, Water Supply |
| description | String | Description of civic maintenance responsibilities and tasks |
| email | String | Official contact email for department supervisory correspondence |
| createdAt, updatedAt | Date | Timestamps tracking department registration and updates |

**Table 4.4 feedbacks collection**

| Field | Type | Description |
|---|---|---|
| _id | ObjectId (Primary Key) | Auto-generated unique feedback submission identifier |
| user | ObjectId (Foreign Key) | References _id in users collection of the citizen submitting review |
| userName | String | Display name of citizen submitting the feedback |
| issue | ObjectId (Foreign Key, nullable) | References _id in issues collection being evaluated |
| issueTitle | String | Title of the resolved grievance referenced by the review |
| rating | Number (1 to 5) | Numerical quality rating (1 to 5 stars) given by citizen |
| category | String | Review category (e.g. 'Resolution Quality', 'Staff Promptness') |
| comment | String | Citizen narrative feedback regarding the completed repair work |
| createdAt, updatedAt | Date | Submission timestamp of citizen review |

## 4.5 Testing

System testing was conducted to verify that every functional module operates according to specifications, that incorrect or incomplete inputs are handled gracefully with informative error messages, and that privileged operations are protected by role-based access rules. Testing was executed on the local development environment using modern web browsers (Google Chrome and Microsoft Edge) across mobile and desktop viewport dimensions.

### 4.5.1 Testing strategy

**Table 4.6 Levels of testing**

| Level | Purpose | Applied to |
|---|---|---|
| Unit testing | Verify individual functions, inputs, and validation rules in isolation | GPS coordinate verification, photo MIME validator, 10-char tracking ID slicer, password validator |
| Integration testing | Check data exchange and coordination between connected modules | Report form to Multer to Cloudinary upload, complaint submission to MongoDB write, admin edit to DB update |
| System testing | Test complete end-to-end user journeys across all roles | Citizen registration -> Issue reporting -> Admin edit -> Department proof upload -> Resolved tracking -> Feedback |
| Security testing | Verify route protection, role boundaries, and input safety | Role-based middleware guards, blocking banned users, preventing non-admin access to edit endpoints |
| Usability testing | Verify responsive UI layout, modals, and user messages | Toast notifications, modal popups, image lightboxes, mobile viewport responsiveness, GPS prompt alerts |

| Level | Purpose | Applied to |
|---|---|---|
| Performance observation | Measure cloud upload latency and database query speeds | Cloudinary buffer streaming response times, issue listing queries, and tracking lookups |

### 4.5.2 Test environment

**Table 4.7 Test environment**

| Item | Details |
|---|---|
| Operating system | Windows 11 Home / Pro (64-bit) |
| Runtime and backend | Node.js v24.19.0, Express.js 5.2.1 |
| Database | MongoDB v7.0 with Mongoose ODM 9.9.4 |
| Frontend environment | React 19.2.7, Vite 8.1.1, React Router 7.18.2 |
| Browsers tested | Google Chrome 124+, Microsoft Edge 124+ |
| Test accounts | Admin: admin@gmail.com; Sanitation: sanitation@gmail.com; Roads: roads@gmail.com; Water: water@gmail.com; Citizen: safwana@gmail.com |

### 4.5.3 Test cases

The executed functional test cases and observed outcomes are detailed in Table 4.8.

**Table 4.8 Test cases**

| ID | Module | Test case | Expected result | Status |
|---|---|---|---|---|
| T01 | Registration | Register with valid name, email, and password | Account is created and login screen is displayed | Pass |
| T02 | Registration | Register with an email already registered in system | Validation error message shown; registration is blocked | Pass |
| T03 | Registration | Register with password shorter than 6 characters | Password complexity error shown; account not created | Pass |
| T04 | Login | Log in with correct registered email and password | Authentication succeeds and user redirected to dashboard | Pass |
| T05 | Login | Log in with incorrect password | Error toast shown ('Invalid credentials'); access denied | Pass |
| T06 | Login | Log in using a blocked account | Login rejected with message ('Account is suspended') | Pass |
| T07 | Logout | Click Logout button in navigation bar | Session cleared and user returned to home/login page | Pass |
| T08 | Report Issue | Submit complaint with both live GPS and photo attached | Complaint saved; unique 10-char tracking ID generated | Pass |
| T09 | Report Issue | Attempt submission without clicking 'Get GPS Location' | Submit blocked with warning toast ('Please get GPS location') | Pass |

| ID | Module | Test case | Expected result | Status |
|---|---|---|---|---|
| T10 | Report Issue | Attempt submission without selecting a photo | Submit blocked with warning toast ('Please attach a photo') | Pass |
| T11 | Report Issue | Upload non-image file (e.g. PDF or text file) | Multer filter rejects file with invalid MIME type error | Pass |
| T12 | Report Issue | Upload image exceeding 10 MB size limit | File size error displayed; file upload rejected | Pass |
| T13 | Report Issue | Browser GPS permission denied by citizen | Warning toast displayed; approximate coordinates set with flag | Pass |
| T14 | Tracking | Search complaint using valid 10-char tracking ID | Complaint record found and tracking timeline displayed | Pass |
| T15 | Tracking | Search complaint using non-existent tracking ID | Error message shown ('No complaint found with this ID') | Pass |
| T16 | Tracking | View milestone stepper for complaint in 'Pending' state | First milestone 'Pending' highlighted in emerald; rest gray | Pass |
| T17 | Tracking | View milestone stepper for complaint in 'In Progress' state | Milestones 'Pending', 'Verified', 'In Progress' active | Pass |
| T18 | Tracking | Click on problem photo thumbnail | High-resolution lightbox modal opens showing photo proof | Pass |
| T19 | Admin Login | Access /admin routes with administrator credentials | Admin dashboard opens with operational statistics | Pass |
| T20 | Admin Access | Citizen attempts direct access to /admin URL | Protected route redirects to login / unauthorized error | Pass |
| T21 | Admin Triage | Admin views list of all submitted civic grievances | All complaints displayed with GPS, photo, and status | Pass |
| T22 | Admin Edit | Admin opens Edit Complaint modal before assignment | Modal opens allowing modification of category, status, notes | Pass |
| T23 | Admin Edit | Admin updates category from 'General' to 'Roads & Infra' | Issue updated in database; category reflects in table | Pass |
| T24 | Admin Edit | Admin assigns issue to 'Sanitation & Waste Management' | Department ID updated; complaint appears in sanitation queue | Pass |
| T25 | Admin Users | Admin toggles account status to Block a citizen user | isBlocked set to true; citizen cannot authenticate | Pass |
| T26 | Admin Depts | Admin adds a new municipal maintenance department | New department saved and appears in department selector | Pass |
| T27 | Dept Portal | Sanitation officer logs in at /department/dashboard | Only grievances assigned to Sanitation department listed | Pass |
| T28 | Dept Portal | Officer transitions issue status to 'In Progress' | Status updated; citizen milestone stepper updates live | Pass |
| T29 | Dept Portal | Officer marks 'Resolved' and uploads resolution photo | Photo uploaded to Cloudinary; issue marked Resolved | Pass |

| ID | Module | Test case | Expected result | Status |
|---|---|---|---|---|
| T30 | Feedback | Citizen opens feedback modal on Resolved complaint | Modal displays 5-star rating selector and comments area | Pass |
| T31 | Feedback | Citizen submits 5-star review with satisfactory comment | Feedback saved and linked to issue and citizen profile | Pass |
| T32 | Feedback | Admin opens Feedbacks tab in administrative dashboard | Submitted citizen reviews and star ratings displayed clearly | Pass |

### 4.5.4 Test summary

The consolidated testing summary across all functional subsystems is presented in Table 4.9.

**Table 4.9 Test summary**

| Part of the system | Test cases | Passed | Failed |
|---|---|---|---|
| Authentication & User Accounts | 7 | 7 | 0 |
| Civic Issue Reporting & GPS/Photo Validation | 6 | 6 | 0 |
| Issue Tracking & Milestone Stepper | 5 | 5 | 0 |
| Administrative Dashboard & Pre-Assignment Editing | 8 | 8 | 0 |
| Department Operations & Resolution Proof | 3 | 3 | 0 |
| Citizen Feedback & Service Rating | 3 | 3 | 0 |
| Total | 32 | 32 | 0 |

# Chapter 5

# Conclusion

FixMyCity demonstrates that modern full-stack web technologies can effectively resolve long-standing operational challenges in municipal civic grievance redressal. By combining React 19, Node.js, Express 5, MongoDB, HTML5 Geolocation, and Cloudinary cloud storage, the system establishes an accountable, transparent, and direct bridge connecting citizens, municipal administrators, and field engineering departments.

A key achievement of the platform is its strict evidentiary integrity: by requiring both live GPS coordinates and photographic proof before an issue can be submitted, the platform eliminates ambiguous, fake, or unverifiable complaints. Furthermore, the administrative capability to edit complaint details prior to department assignment guarantees accurate classification and prevents misplaced municipal effort. The four-stage tracking stepper provides complete visibility into complaint progress, and the requirement for department officers to upload photographic completion proof ensures that reported defects are genuinely repaired before tickets are closed.

The system was engineered across five Agile Scrum sprints, incorporating role-based access control, secure credential handling, input validation, and cloud asset storage. FixMyCity provides urban local bodies and ward councillors with a dependable, cost-free foundation that can be readily deployed to improve urban living conditions and citizen engagement.

## 5.1 Future Scope

• AI Automated Issue Classification: Integrating computer vision models to automatically detect and classify defects (e.g. pothole depth, garbage volume, streetlight damage) directly from uploaded images.

• Native Mobile Applications: Developing native Android and iOS applications with background offline caching to allow citizens to capture issues in areas with intermittent internet connectivity.

• WhatsApp and Telegram Bot Integration: Enabling citizens to report issues and receive milestone status notifications via popular messaging apps.

• Automated SMS and Email Alerts: Dispatching real-time notifications to citizens and ward councillors whenever an issue status transitions.

• GIS Municipal Heatmaps: Providing spatial mapping dashboards displaying complaint clusters to assist municipal planners in allocating maintenance budgets.

• Public Drone Inspection Feeds: Allowing integration of aerial surveillance feeds for monitoring major roadway networks and canal drainage corridors.

## 5.2 References

1. World Bank. (2021). Technology and Civic Engagement in Municipal Service Delivery. World Bank Urban Development Series.

2. Open Geospatial Consortium (OGC). (2020). W3C Geolocation API Specification (2nd Edition). World Wide Web Consortium.

3. Pressman, R. S., & Maxim, B. R. (2020). Software Engineering: A Practitioner's Approach (9th Edition). McGraw-Hill Education.

4. Schwaber, K., & Sutherland, J. (2020). The Scrum Guide: The Definitive Guide to Scrum: The Rules of the Game. Scrum.org.

5. Elmasri, R., & Navathe, S. B. (2017). Fundamentals of Database Systems (7th Edition). Pearson Education.

6. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). Database System Concepts (7th Edition). McGraw-Hill Education.

7. Banker, K., Bakkum, P., Verch, S., Garrett, D., & Hawkins, T. (2016). MongoDB in Action: Principles and Practices of Document Databases (2nd Edition). Manning Publications.

8. Flanagan, D. (2020). JavaScript: The Definitive Guide: Master the World's Most-Used Programming Language (7th Edition). O'Reilly Media.

9. Banks, A., & Porcello, E. (2020). Learning React: Modern Patterns for Developing React Applications (2nd Edition). O'Reilly Media.

10. Cantelon, M., Harter, M., Holowaychuk, T. J., & Rajlich, N. (2017). Node.js in Action (2nd Edition). Manning Publications.

11. Cloudinary Ltd. (2024). Cloudinary API Documentation: Programmable Media and Asset Management. Retrieved from: https://cloudinary.com/documentation

12. Express.js Foundation. (2024). Express.js Framework Documentation (v5.x). Retrieved from: https://expressjs.com/

13. MongoDB Inc. (2024). Mongoose ODM Documentation (v9.x). Retrieved from: https://mongoosejs.com/docs/

14. Vite Core Team. (2024). Vite: Next Generation Frontend Tooling Documentation. Retrieved from: https://vitejs.dev/

15. Mozilla Developer Network (MDN). (2024). Using the Geolocation API. Retrieved from: https://developer.mozilla.org/en-US/docs/Web/API/Geolocation_API

16. OpenStreetMap Foundation. (2024). Nominatim Reverse Geocoding API Guidelines. Retrieved from: https://nominatim.org/

# Chapter 6

# Appendix

This appendix presents key implementation source code excerpts from the FixMyCity application.

## 6.1 Source Code

**Database Model: Issue Schema with Live Coordinates & Tracking ID**

```
// backend/models/Issue.js - Issue Schema with GPS Coordinates & Tracking ID
const mongoose = require("mongoose");

const issueSchema = new mongoose.Schema(
  {
    title: { type: String, required: true },
    category: { type: String, default: "General" },
    description: { type: String, default: "" },
    latitude: { type: Number, default: 0 },
    longitude: { type: Number, default: 0 },
    photo: { type: String, default: null },
    resolvedPhoto: { type: String, default: null },
    trackingId: { type: String, trim: true, index: true },
    status: {
      type: String,
      enum: ["Pending", "Verified", "In Progress", "Resolved", "Rejected"],
      default: "Pending",
    },
    department: { type: mongoose.Schema.Types.ObjectId, ref: "Department", default: null
},
    user: { type: mongoose.Schema.Types.ObjectId, ref: "User", default: null },
    feedback: {
      rating: { type: Number, min: 1, max: 5, default: null },
      comment: { type: String, default: "" },
      submittedAt: { type: Date, default: null },
    },
  },
  { timestamps: true }
);

issueSchema.pre("save", function () {
  if (!this.trackingId && this._id) {
    this.trackingId = this._id.toString().slice(0, 10);
  }
});

module.exports = mongoose.model("Issue", issueSchema);
```

**REST Controller: Civic Issue Reporting with Mandatory Photo & GPS Validation**

```
// backend/server.js - Civic Issue Reporting with Mandatory Photo & GPS Validation
app.post("/api/issues", upload.single("photo"), async (req, res) => {
  try {
    const { title, category, description, latitude, longitude, department } = req.body;
    const userId = req.headers["user-id"];

    // Mandatory Photo Validation
    if (!req.file) {
```

```
      return res.status(400).json({ message: "A photo of the civic defect is mandatory."
});
    }

    // Mandatory GPS Location Validation
    const lat = Number(latitude);
    const lng = Number(longitude);
    if (!lat || !lng || isNaN(lat) || isNaN(lng)) {
      return res.status(400).json({ message: "Valid live GPS coordinates are required."
});
    }

    // Stream image buffer directly to Cloudinary
    const photoUrl = await uploadToCloudinary(req.file.buffer,
"fixmycity/citizen_proofs");

    const issue = new Issue({
      title,
      category: category || "General",
      description,
      latitude: lat,
      longitude: lng,
      photo: photoUrl,
      status: "Pending",
      department: department && mongoose.Types.ObjectId.isValid(department) ? department
: null,
      user: userId && mongoose.Types.ObjectId.isValid(userId) ? userId : null,
    });

    await issue.save();
    return res.status(201).json({
      message: "Issue reported successfully.",
      trackingId: issue.trackingId,
      issue,
    });
  } catch (error) {
    return res.status(500).json({ message: "Failed to report issue: " + error.message });
  }
});
```

**REST Controller: Administrative Pre-Assignment Issue Editing**

```
// backend/server.js - Administrative Pre-Assignment Issue Editing
app.put("/api/issues/:id", async (req, res) => {
  try {
    const { id } = req.params;
    const { title, category, description, status, department } = req.body;

    if (!id || !mongoose.Types.ObjectId.isValid(id)) {
      return res.status(400).json({ message: "Invalid complaint ID format." });
    }

    const issue = await Issue.findById(id);
    if (!issue) {
      return res.status(404).json({ message: "Complaint not found." });
    }

    if (issue.status === "Resolved") {
      return res.status(400).json({ message: "Resolved complaints are closed and cannot
be edited." });
    }

    // Verify requesting user has administrator privileges
```

```
    const userId = req.headers["user-id"];
    const requester = await User.findById(userId);
    if (!requester || requester.role !== "admin") {
      return res.status(403).json({ message: "Access denied. Only administrators can edit
issue details." });
    }

    // Apply administrative modifications prior to department allocation
    if (category) issue.category = category.trim();
    if (description !== undefined) issue.description = description;
    if (status) issue.status = status;
    if (department) {
      issue.department = mongoose.Types.ObjectId.isValid(department) ? department : null;
      if (issue.status === "Pending") issue.status = "Verified";
    }

    await issue.save();
    return res.json({ message: "Complaint updated successfully.", issue });
  } catch (error) {
    return res.status(500).json({ message: "Error updating complaint: " + error.message
});
  }
});
```

**REST Controller: Department Work Resolution Proof Upload**

```
// backend/server.js - Department Work Resolution Proof Upload
app.put("/api/issues/:id/resolved-photo", upload.single("resolvedPhoto"), async (req,
res) => {
  try {
    const { id } = req.params;
    if (!req.file) {
      return res.status(400).json({ message: "Resolution work completion photograph is
mandatory." });
    }

    const issue = await Issue.findById(id);
    if (!issue) return res.status(404).json({ message: "Complaint record not found." });

    // Stream resolution proof image buffer to Cloudinary
    const proofUrl = await uploadToCloudinary(req.file.buffer,
"fixmycity/resolved_proofs");

    issue.resolvedPhoto = proofUrl;
    issue.status = "Resolved";
    await issue.save();

    return res.json({ message: "Resolution proof uploaded and issue closed as Resolved.",
issue });
  } catch (error) {
    return res.status(500).json({ message: "Error uploading resolution proof: " +
error.message });
  }
});
```

**Client Component: HTML5 GPS Auto-Detection & Dual Validation (ReportIssue.jsx)**

```
// frontend/src/pages/ReportIssue.jsx - GPS Auto-Detection & Dual Validation
const getCurrentLocation = () => {
  if (!navigator.geolocation) {
```

```
    showToast("GPS location is not supported by your browser.", "error");
    return;
  }
  setLocating(true);
  navigator.geolocation.getCurrentPosition(
    (position) => {
      setLocating(false);
      setLocation({
        latitude: position.coords.latitude,
        longitude: position.coords.longitude,
      });
      showToast("Live GPS coordinates detected successfully!", "success");
    },
    (error) => {
      setLocating(false);
      showToast("GPS permission denied. Please allow location access to submit.",
"error");
    },
    { enableHighAccuracy: true, timeout: 10000 }
  );
};

const handleSubmit = async (e) => {
  e.preventDefault();
  // Enforce mandatory Dual Validation (Live GPS + Photo)
  if (!location) {
    showToast("Mandatory requirement: Please acquire live GPS location.", "warning");
    return;
  }
  if (!photo) {
    showToast("Mandatory requirement: Please attach an evidentiary photo.", "warning");
    return;
  }

  const formData = new FormData();
  formData.append("title", title);
  formData.append("description", description);
  formData.append("latitude", location.latitude);
  formData.append("longitude", location.longitude);
  formData.append("photo", photo);

  const response = await fetch(`${API_BASE_URL}/api/issues`, {
    method: "POST",
    headers: getAuthHeaders(),
    body: formData,
  });
  const data = await response.json();
  if (response.ok) {
    setTrackingId(data.trackingId);
    setSubmitted(true);
  }
};
```

### Client Component: 4-Stage Milestone Tracking Stepper (TrackComplaint.jsx)

```
// frontend/src/pages/TrackComplaint.jsx - 4-Stage Milestone Stepper
const TRACKING_STEPS = [
  { key: "Pending", label: "Pending", desc: "Reported in Portal" },
  { key: "Verified", label: "Verified", desc: "Admin Inspected" },
  { key: "In Progress", label: "In Progress", desc: "Field Team Dispatched" },
  { key: "Resolved", label: "Resolved", desc: "Work Completed & Verified" },
];
```

```
function getStepIndex(status) {
  const map = { Pending: 0, Verified: 1, "In Progress": 2, Resolved: 3 };
  return map[status] !== undefined ? map[status] : 0;
}

// Milestone Stepper Render
<div className="stepper-container">
  {TRACKING_STEPS.map((step, idx) => {
    const isCompleted = idx <= getStepIndex(currentIssue.status);
    const isCurrent = idx === getStepIndex(currentIssue.status);
    return (
      <div key={step.key} className={`step-node ${isCompleted ? "active" : ""}`}>
        <div className={`step-circle ${isCurrent ? "current" : ""}`}>{idx + 1}</div>
        <span className="step-label">{step.label}</span>
        <span className="step-desc">{step.desc}</span>
      </div>
    );
  })}
</div>
```